App Móvil para el Reconocimiento y Gestión de Información Nutricional a partir de Etiquetas de Productos

Autor:
Guillermo Efrén Medina Guamán
Director/Directora:
Jordi Marco Gómez
Titulación:
Grado en Ingeniería de Sistemas Audiovisuales
Convocatoria:
Primavera, 2025

Índice

1. Introducción
2. Estado de la tecnología
3. Metodología
4. Desarrollo de la aplicación
5. Resumen de presupuesto
6. Implicaciones sociales y conclusiones

1. Introducción

¿Porque una aplicación de nutrición?

¿Cual es el objetivo final?

2.1 Estudio de mercado

Característica	Yuka	Fitia	Yazio	Virtuagym
Detalle nutricional completo	Si	Si	No	No
Registro diario de alimentos	No	Si	No	Si
Introducción rápida de datos	No	No	No	Si
Gratuita	Si	No	No	Si
Base de datos precisa	Si	No	No	No
Uso de AI	No	No	No	No

- Tendencia emergente: Cal AI

2.2 Estudio de Inteligencia Artificial

Comparativa de modelos de IA utilizados para el registro automático de nutrientes.

Característica	Mistral	google vision	DeepSeek	Gpt4o
Reconocimiento OCR	Bajo	Excelente	Bueno	Excelente
Rendimiento (velocidad)	Media	Excelente	Media	Alta
Precisión de la respuesta	Mala	Mala	Alta	Excelente
Costo	Bajo	Medio	Bajo	Alto

GPT-4o fue elegido por su precisión, velocidad y capacidad OCR en un solo modelo.

2.3 Tecnologías utilizadas

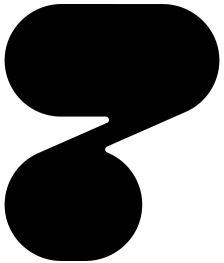
Principales tecnologías y herramientas empleadas

Visual Studio Code

Firebase

Flutter

ChatGPT



HTTPie

3. Metodología

A continuación se describen las etapas para estructurar el desarrollo del proyecto:

- Definición de funcionalidades
  - Registro inteligente de alimentos mediante imagen
  - Registro manual de alimentos
  - Cálculo automático de nutrientes
  - Planificación de comidas
  - Seguimiento del consumo de alimentos
- Diseño de la arquitectura del sistema
  - **Frontend:** Representa la interfaz con la que interactúa el usuario.
  - **Backend:** Es el encargado del procesamiento de datos y la lógica del sistema.
  - **Base de datos:** Es donde se almacena toda la información nutricional, de usuario y planificaciones.
  - **Módulo de Inteligencia Artificial:** El cual permite el reconocimiento de alimentos y etiquetas a partir de imágenes tomadas por el usuario.

4.1 Investigación sobre modelos de Inteligencia Artificial

- ¿Qué es un prompt?
- ¿Cómo funciona un Prompt?
- ¿Qué valores se tuvieron en cuenta?

```
{
  "model": "gpt-4o",
  "messages": [
    {
      "role": "system",
      "content": "..."
    },
    {
      "role": "user",
      "content": "..."
    }
  ]
}
```

4.1 Investigación sobre modelos de Inteligencia Artificial

- DeepSeek:


```
"image": {
  "content":"/9j/4AAQSkZJRgABAQEASABIAAD/4QPQRXhpZg
  AATUOAKgAAAAgABAEPAAIAAAAIAAABSgEQAAIAAAAJAAA
  BUodpAAQAAAABAAABXOocAAcAAAEMAAAAPgAAAAAc6gAAAAEA
  ... asi hasta 893014 caracteres
},
```


- ChatPT-4o:


```
"image_url": {
  "url": "https://firebasestorage.googleapis.com/v0/b/
  nutritionapplication23b5c.firebasestorage.app/o/
  capturas%2Fcaptura_8L9g7TiugQUC2z5UY26x_1748443467201.
  jpg?alt=media&token=9f0d3ae8-7499-4864-94dc-b6831c7843f9"
}
```

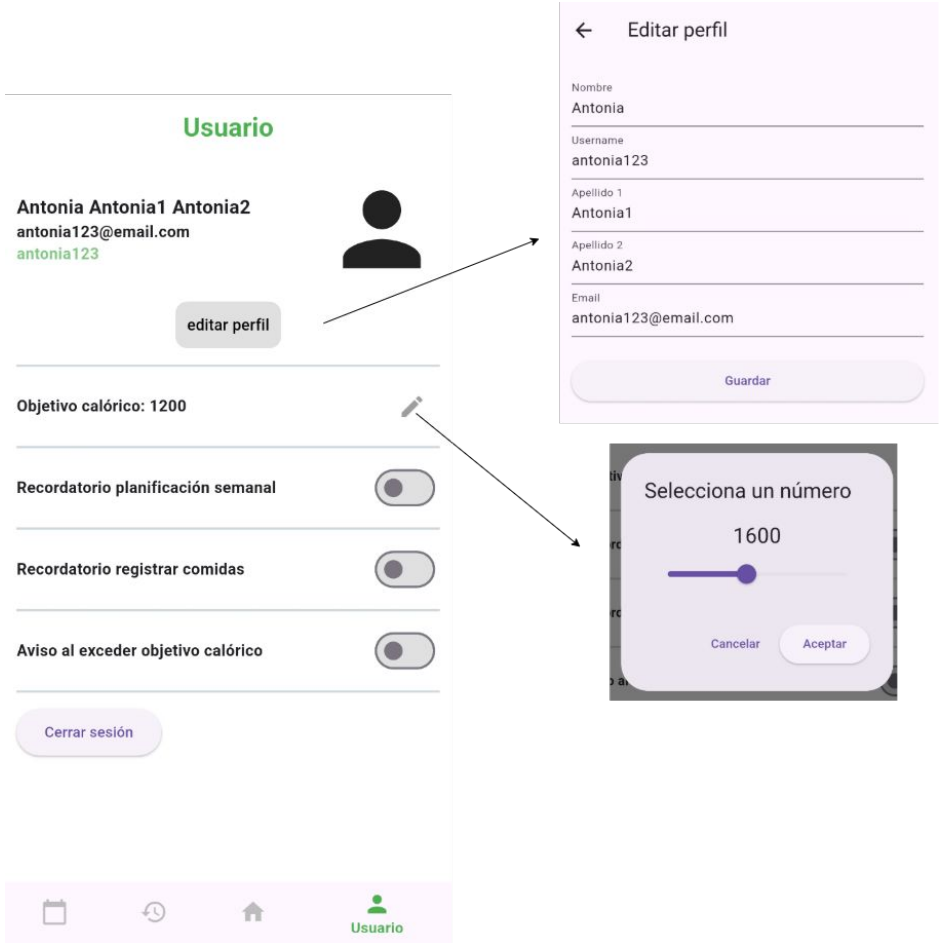

- Google-vision + ChatPT-4o:


```
"content": "Aqu´ı tienes el texto extra´ıdo de la
imagen con el OCR de Google: ’KAROT\nTTE, SUB\nEL).
\nDURCHSCHNITTLICHE N¨AHRWERTE\n100 ml\n500 ml (%*)
\nENERGIE\n203 kJ/\n1014 kJ/\n→\nKOHLENHYDRATE\n
(48 kcal)\n(239 kcal) (12%)\nDAVON ZUCKER\n12 g\n60
g (23%)\nSALZ\n11 g\n56 g (62%)\n0,05 g\n0,24 g (4%
)\nVITAMINE\n100 ml (%**)\nVITAMIN B6 0,80 mg (57%)
4,0 mg (286%)\nRIBOFLAVIN 0,70 mg (50%) 3,5 mg (250%)
\nVITAMIN B12\n2,5 µg (100%)\n13 µg (500%)\nENTHALT
GERINCE CICE MENGEN VON FETT,\n500 ml (%**)\n43 mg
(266%)’. Extrae los valores nutricionales y
devu´elvelos en JSON, utilizando el idioma espa~nol.
No uses formato de c´odigo, solo el JSON limpio."
```

4.2 Frontend y Diseño y desarrollo de la interfaz de usuario

Pantalla Usuario:

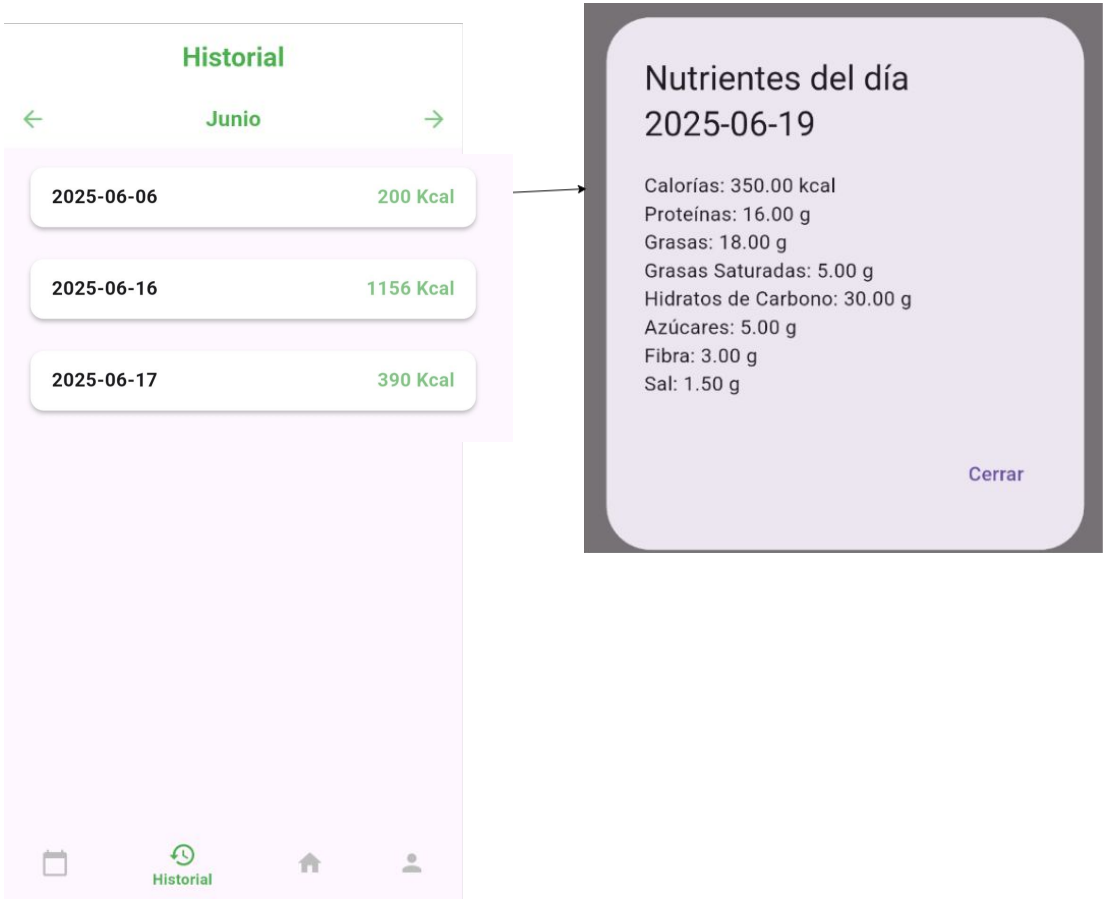
- Presenta información de usuario
- Ajustar objetivo calórico
- Editar opciones de usuario.



4.2 Frontend y Diseño y desarrollo de la interfaz de usuario

Pantalla Historial:

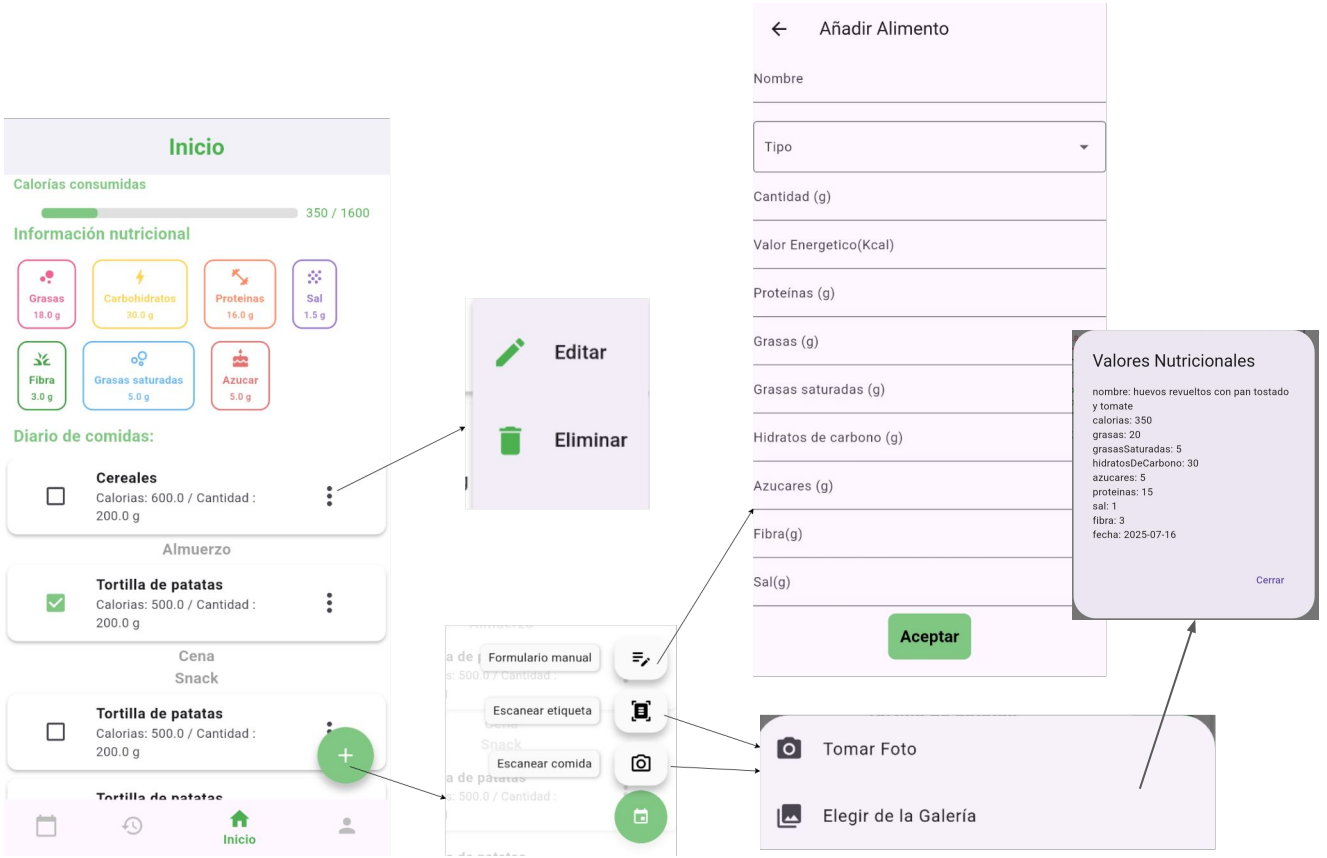
- Despliegue visual y organizado
- Resumen de nutrientes totales



4.2 Frontend y Diseño y desarrollo de la interfaz de usuario

Pantalla Home:

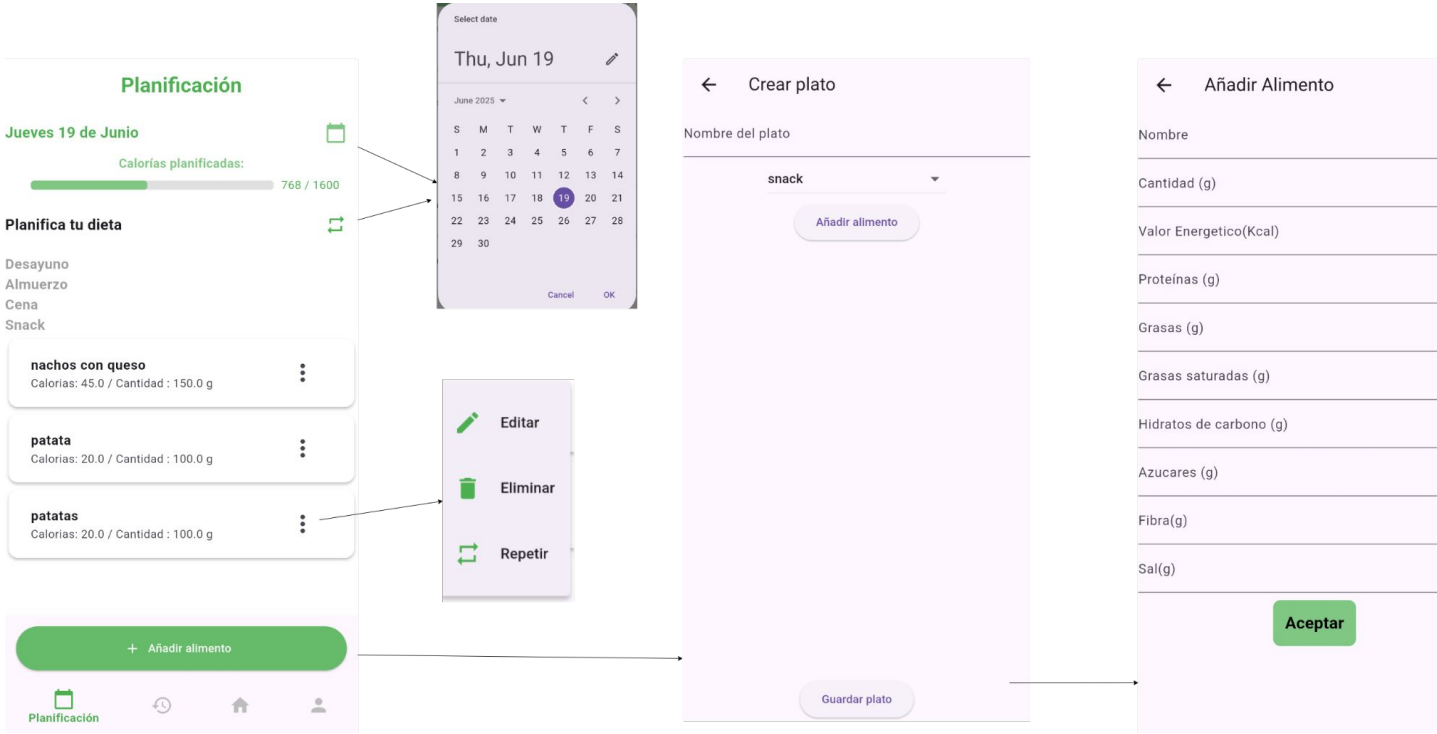
- Progreso calórico
- Resumen nutricional
- Planificación
- Botón registro



4.2 Frontend y Diseño y desarrollo de la interfaz de usuario

Pantalla Planificación:

- Calendario
- Visual alimentos
- Botón añadir alimentos



4.3 Backend y Base de datos

Modelos de datos:

- Atributos:

```
class Usuario{
  String? id;
  String nombre;
  String username;
  String apellido1;
  String? apellido2;
  String email;
  String imagen;
  num objetivoCalorico;
  bool recordatorioComidas;
  bool recordatorioPlanificacion;
  bool avisoExcesoCalorias;
  String? capturaUrl;
```

- Constructor:

```
Usuario(
  {this.id,
  required this.nombre,
  required this.username,
  required this.apellido1,
  this.apellido2,
  required this.email,
  this.imagen = '',
  required this.objetivoCalorico,
  this.recordatorioComidas = false,
  this.recordatorioPlanificacion = false,
  this.avisoExcesoCalorias = false,
  this.capturaUrl = '',
  });
```

4.3 Backend y Base de datos

Modelos de datos:

- toJson():

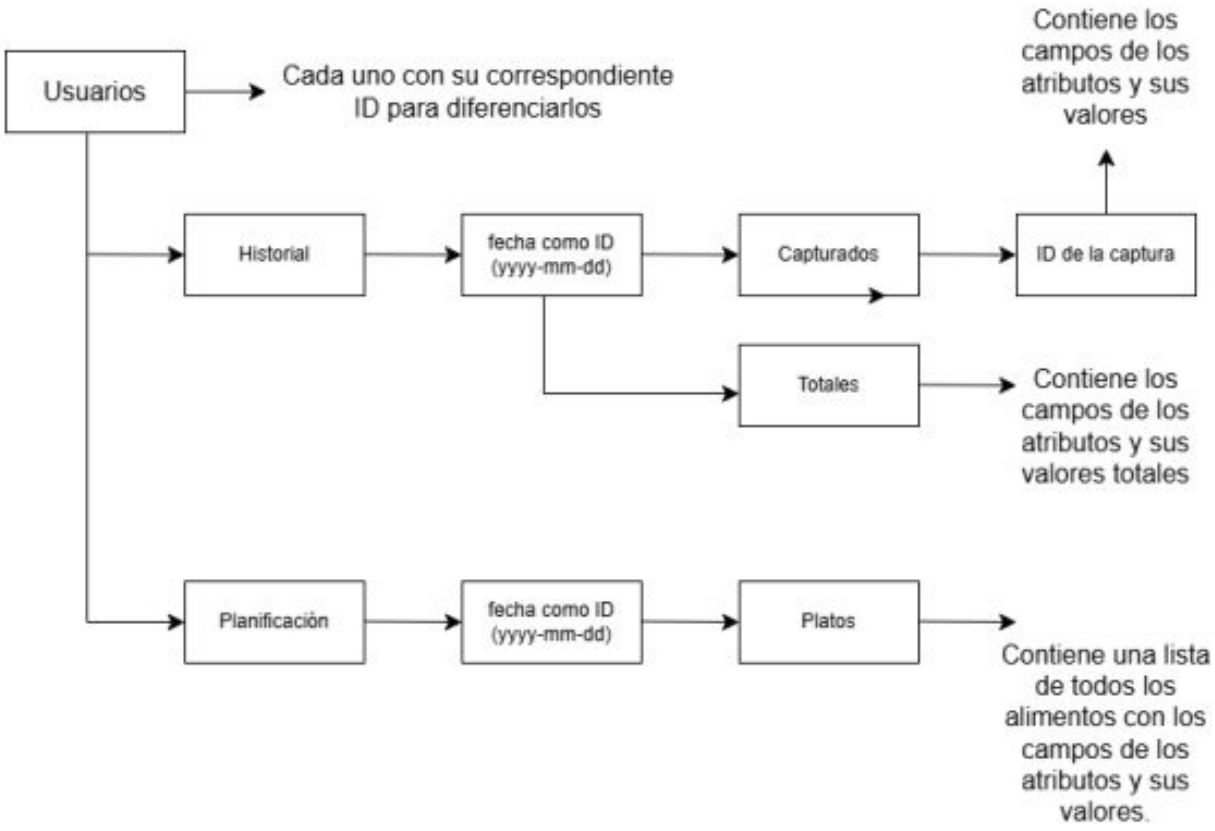
```
Map<String, dynamic> toJson() => {
  'nombre': nombre,
  'username': username,
  'apellido1': apellido1,
  'apellido2': apellido2,
  'email': email,
  'imagen': imagen,
  'objetivoCalorico': objetivoCalorico,
  'recordatorioComidas': recordatorioComidas,
  'recordatorioPlanificacion': recordatorioPlanificacion,
  'avisoExcesoCalorias': avisoExcesoCalorias,
  'capturaUrl': capturaUrl,
```

- fromJson():

```
factory Usuario.fromJson(Map<String, dynamic> json, {String? id}) {
  return Usuario(
    id: id ?? json['id'],
    nombre: json['nombre'] ?? '',
    username: json['username'] ?? '',
    apellido1: json['apellido1'] ?? '',
    apellido2: json['apellido2'],
    email: json['email'] ?? '',
    imagen: json['imagen'] ?? '',
    objetivoCalorico: json['objetivoCalorico'] ?? 0,
    recordatorioComidas: json['recordatorioComidas'] ?? false,
    recordatorioPlanificacion: json['recordatorioPlanificacion'] ?? false,
    avisoExcesoCalorias: json['avisoExcesoCalorias'] ?? false,
    capturaUrl: json['capturaUrl'],
  );
}
```

4.3 Backend y Base de datos

Diagrama ilustrativo de las colecciones de la base de datos:



4.3 Backend y Base de datos

Gestión de datos: ***"CRUD"***

- Crear:

```
//Crear historial vacio en firebase.
  Future<void> crearHistorialDia(DateTime fecha) async {
    final String fechaId = _formatoFecha(fecha);
    final DocumentReference ref = _db
        .collection('usuarios')
        .doc(userId)
        .collection('historial')
        .doc(fechaId);

    final Map<String, dynamic> historialInicial = {
    'fecha': fechaId,
    'totales': NutrientesTotales().toJson(),
  };

    await ref.set(historialInicial, SetOptions(merge: true));
  }
```

- Eliminar:

```
Future<void> eliminarRegistroDeHistorial({
required DateTime fecha,
Alimento? alimento,
 PlatoPlanificado? plato,
}) async {
  final String fechaId = _formatoFecha(fecha);
  final DocumentReference ref = _db
      .collection('usuarios')
      .doc(userId)
      .collection('historial')
      .doc(fechaId);

  final snapshot = await ref.get();

  if (!snapshot.exists) return;

  if (alimento != null) {
    await ref.collection('capturados').doc(alimento.id).delete();
  }

  if (plato != null) {
    await ref.collection('platosConsumidos').doc(plato.id).delete();
  }

  await actualizarTotales(fecha);
}
```

4.3 Backend y Base de datos

Gestión de datos: ***"CRUD"***

- Leer:

```
Future<Historial?> getHistorialPorFecha(DateTime fecha) async {
  final String fechaId = _formatoFecha(fecha);
  final DocumentReference ref = _db
      .collection('usuarios')
      .doc(userId)
      .collection('historial')
      .doc(fechaId);

  final snapshot = await ref.get();
  if (!snapshot.exists) return null;

  final data = snapshot.data() as Map<String, dynamic>;
```

```
  // Obtener alimentos capturados
  final capturadosSnapshot = await ref.collection
  ('capturados').get();
  final capturados = capturadosSnapshot.docs.map((doc) {
    final alimento = Alimento.fromJson(doc.data());
    alimento.id = doc.id;
    return alimento;
  }).toList();
  // Obtener platos seleccionados
  final platosSnapshot = await ref.collection
  ('platosConsumidos').get();
  final platos = platosSnapshot.docs.map((doc) {
    return PlatoPlanificado.fromJson(doc.data(), doc.id);
  }).toList();
  // Crear historial
  final historial = Historial.fromJson(data);
  historial.capturados = capturados;
  historial.platosConsumidos = platos;

  return historial;
}
```

4.3 Backend y Base de datos

Gestión de datos: ***"CRUD"***

- Actualizar:

```
Future<void> registroAlimento({
  required DateTime fecha,
  Alimento? alimento,
  PlatoPlanificado? plato,
}) async {
  final String fechaId = _formatoFecha(fecha);
final DocumentReference diaRef = _db
    .collection('usuarios')
    .doc(userId)
    .collection('historial')
    .doc(fechaId);

final docSnapshot = await diaRef.get();
```

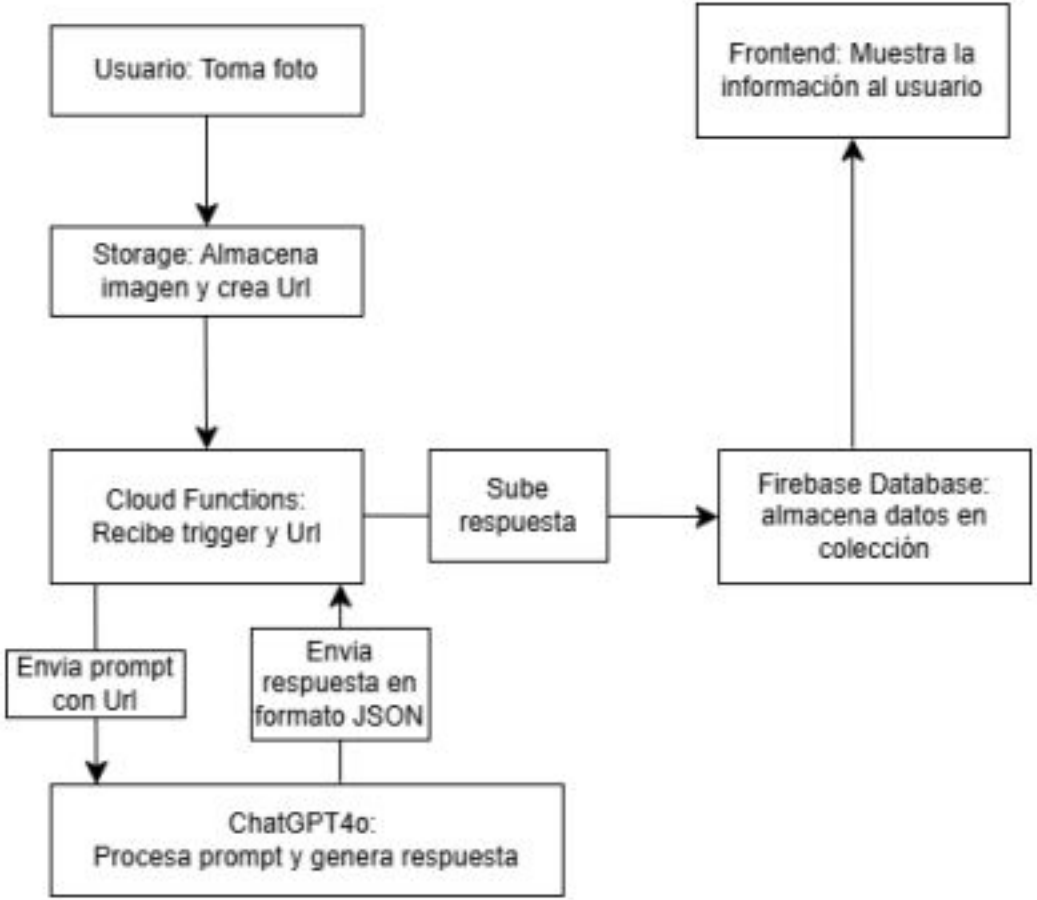
```
if (!docSnapshot.exists) {
  await crearHistorialDia(fecha);
} else {
  final data = docSnapshot.data() as Map<String, dynamic>? ?? {};
  if (!data.containsKey('fecha')) {
    await diaRef.set({'fecha': fechaId}, SetOptions(merge: true));
  }
}

if (alimento != null) {
  final CollectionReference capturadosRef = diaRef.collection('capturados');
  final docRef = capturadosRef.doc();
  alimento.id = docRef.id;
  await docRef.set(alimento.toJson());
}

if (plato != null) {
  final CollectionReference platosRef = diaRef.collection('platosConsumidos')
  final docRef = platosRef.doc(plato.id);
  await docRef.set(plato.toJson());
}

  await actualizarTotales(fecha);
}
```

4.4 Implementación de Inteligencia Artificial



4.4 Implementación de Inteligencia Artificial

- Tomar foto:

```
Future<Map<String, dynamic>?> procesarImagenEnCloud(
  String imageUrl, String tipo, String userId, String fecha) async {
final url = Uri.parse('https://us-central1-nutrition
application-23b5c.cloudfunctions.net/procesarComida');
final body = jsonEncode({'userId': userId,
'imageUrl': imageUrl, 'tipo': tipo,'fecha': fecha,});
final response = await http.post(
  url,
  headers: {'Content-Type': 'application/json'},
  body: body,
);
if (response.statusCode == 200) {
  final data = jsonDecode(response.body);
  return data['data'] as Map<String, dynamic>?;
} else {
  print('Error en la solicitud: ${response.statusCode} - ${response.body}');
  return null;
}
```

4.4 Implementación de Inteligencia Artificial

- Cloud function:

```
        async function pedirRespuestaOpenAI(messages) {
  const response = await fetch("https://api.openai.com/v1/chat/completions", {
    method: "POST",
    headers: {
      "Authorization": `Bearer ${OPENAI_API_KEY}`,
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      model: "gpt-4o",
      messages,
    }),
  });
  const data = await response.json();
  return data.choices[0].message.content.trim();
}
...
const messages = [
      { role: "system", content: systemMessageContent },
      {
        role: "user",
        content: [
          { type: "text", text: "Extrae los valores nutricionales y devuelve el JSON." },
          { type: "image_url", image_url: { url: imageUrl } },
        ],
      },
```

4.4 Implementación de Inteligencia Artificial

- Prompt etiqueta:

```
Eres un experto en nutrición. Analiza la imagen de una etiqueta de información nutricional
(formato de producto industrial).Devuelve solo un JSON con las siguientes claves exactas
(minúsculas, con guiones bajos) y sus valores por 100g o por ración, si se indica:
{
  "nombre": "string con el nombre del alimento",
  "calorias": número,

  "grasas": número,
  "grasasSaturadas": número,
  "hidratosDeCarbono": número,
  "azucares": número,
  "proteinas": número,
  "sal": número,
  "fibra": número
}
Si algún dato no aparece, indícalo como null. No añadas texto extra ni explicaciones.
Solo JSON.
```

- Prompt captura de la comida:

```
Eres un nutricionista experto. Estás viendo una imagen de un plato de comida.
Estima los valores nutricionales aproximados del plato. Devuelve solo un JSON
con las siguientes claves exactas y sus valores estimados por ración:
{
  "nombre": "nombre estimado del plato",
  "calorias": número,
  "grasas": número,
  "grasasSaturadas": número,
  "hidratosDeCarbono": número,
  "azucares": número,
  "proteinas": número,
  "sal": número,
  "fibra": número
}
Si no puedes estimar algún valor, pon null.
No añadas ningún texto ni explicaciones. Solo JSON.
```

4.4 Implementación de Inteligencia Artificial

```
const content = await pedirRespuestaOpenAI
(messages);
let cleanContent = content;
if (cleanContent.startsWith("```json")) {
  cleanContent = cleanContent.slice(7).trim();
} else if (cleanContent.startsWith("```")) {
  cleanContent = cleanContent.slice(3).trim();
}
if (cleanContent.endsWith("```")) {
  cleanContent = cleanContent.slice(0, -3).trim();
}
const jsonData = JSON.parse(cleanContent);
```

- Subida de datos:

```
function parseNutrientes(data) {
return {
  nombre: data.nombre || null,
  calorias: typeof data.calorias === "number" ? data.calorias : null,
                                ...
const resultadoNormalizado = parseNutrientes
(jsonData);
  const historialRef = db
  .collection("usuarios")
  .doc(userId)
  .collection("historial")
  .doc(fechaId);
  await historialRef.set({}, { merge: true });
  const nuevoAlimento = {
    id: db.collection("temp").doc().id,
    ...resultadoNormalizado,
  tipoComida: tipo,
  cantidad: 100,
};
```

```
await historialRef.collection("capturados").
doc(nuevoAlimento.id).set(nuevoAlimento);
await actualizarTotales(userId, fechaId);
```

5. Resumen del presupuesto

- Horas de desarrollo:
  - 20 horas a la semana durante 4 meses.
  - A un precio de 15€/h
  - 320 x 15 = 4800€
- Desgaste de materiales:
  - Equipo informático, consumo eléctrico y conexión wifi
  - Precio tarifa wifi : 15-20€(aproximado)
  - Precio aproximado de 85€
- Herramientas de Inteligencia Artificial:
  - Precio promedio de APIs: 7€
  - Precio aproximado: 4 (APIs) x 7€ =28€

Presupuesto total : 4800 + 85 + 28 = 4913€

6. Implicaciones sociales y conclusiones

- Impacto social:
  - Mejora la accesibilidad y la inclusión digital en nutrición.
  - Útil para personas con necesidades dietéticas especiales.
  - Fomenta la educación alimentaria desde la tecnología.
- Conclusiones principales:
  - Cumple objetivos iniciales.
  - Adaptabilidad durante el desarrollo.
  - Posibilidad de ampliación futura: base de datos propia, UI más fluida en la planificación.

Gracias por vuestra atención